Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-214БВ-25

Студент: Степанова А. М.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 05.10.26

Москва, 2026

Постановка задачи

Вариант 19.

Родительский процесс создает два дочерних процесса. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия File с таким именем на запись для child1. Аналогично для второй строки и процесса child2. Родительский и дочерний процесс должны быть представлены разными программами.

Правило фильтрации: с вероятностью 80% строки отправляются в pipe1, иначе в pipe2. Дочерние процессы удаляют все гласные из строк.

Ограничения: допускается использование только системных вызовов, а из стандартной библиотеки С только заголовочные файлы stdio.h, threads.h и stdatomic.h.

Общий метод и алгоритм решения

В лабораторной работе использованы следующие системные вызовы:

- pid_t fork(void); – создает дочерний процесс, являющийся копией родительского.
- int execv(const char *path, char *const argv[]); – заменяет образ текущего процесса образом другой программы (в работе запускается программа child).
- int pipe(int *fd); – создает неименованный канал: fd[0] – конец для чтения, fd[1] – конец для записи.
- int dup2(int oldfd, int newfd); – делает newfd копией дескриптора oldfd (используется для подмены stdin и stdout у дочерних процессов).
- int open(const char *path, int flags, mode_t mode); – открывает файл (в работе – файлы для записи и /dev/urandom для чтения).
- ssize_t read(int fd, void *buf, size_t count); – читает данные из файлового дескриптора.
- ssize_t write(int fd, const void *buf, size_t count); – записывает данные в файловый дескриптор.
- int close(int fd); – закрывает файловый дескриптор.
- pid_t wait(int *status); – ожидает завершения дочернего процесса.
- ssize_t readlink(const char *path, char *buf, size_t size); – читает значение символической ссылки (используется для /proc/self/exe, чтобы найти каталог с программой).

В работе реализованы две программы: parent (родительский процесс) и child (дочерний процесс). Программа child запускается дважды, по одному разу для child1 и child2.

Работа родителя устроена так. Сначала с помощью readlink по пути /proc/self/exe определяется каталог, в котором лежит parent, чтобы затем запустить child из того же каталога. Затем из стандартного потока ввода считываются первые две строки – имена файлов, и оба файла открываются на запись вызовом open с флагами O_WRONLY | O_CREAT | O_TRUNC. После этого создаются два канала pipe1 и pipe2.

Далее родитель дважды вызывает fork. В каждом дочернем процессе закрываются все неиспользуемые концы каналов и чужой файл, затем через dup2 стандартный ввод заменяется на читающий конец своего канала (pipe1 для child1, pipe2 для child2), а стандартный вывод заменяется на дескриптор своего файла. После этого вызывается execv, и процесс превращается в программу child. Благодаря такой подмене программа child ничего не знает ни про каналы, ни про файлы: она просто читает stdin и пишет в stdout.

В родительском процессе закрываются читающие концы каналов и дескрипторы файлов. Затем в цикле считывается каждая следующая строка, и для нее выбирается канал: из /dev/urandom читается один байт, и если его значение по модулю 100 меньше 80, строка записывается в pipe1, иначе – в pipe2. Таким образом, примерно 80% строк попадают к child1 и 20% к child2. Функция rand() из stdlib.h не используется, так как по условию разрешены только системные вызовы и заголовки stdio.h, threads.h и stdatomic.h.

Программа child в цикле читает данные из stdin, копирует в выходной буфер все символы, кроме гласных английского алфавита (a, e, i, o, u в любом регистре), и записывает результат в stdout, то есть в файл. Так как удаление гласных не зависит от соседних символов, собирать строки целиком не требуется: каждый прочитанный блок обрабатывается независимо.

Когда ввод пользователя заканчивается (Ctrl-D), родитель закрывает записывающие концы обоих каналов. Дочерние процессы получают конец файла (read возвращает 0) и завершаются, а родитель дважды вызывает wait, дожидаясь их завершения. Важно, что в каждом процессе закрыты все лишние дескрипторы: если бы child2 унаследовал записывающий конец pipe1, то child1 никогда бы не получил конец файла, и программа зависла бы.

Код программы

parent.c

```
#include <stdint.h>
#include <stdbool.h>

#include <unistd.h>
#include <sys/wait.h>
#include <fcntl.h>
#include <stdlib.h>
#include <stdio.h>

static char CLIENT_PROGRAM_NAME[] = "child";

int read_line(char *buf, int max) {
	int i = 0;
	char c;
	while (i < max - 1 && read(STDIN_FILENO, &c, 1) > 0) {
		buf[i++] = c;
		if (c == '\n') {
			break;
		}
	}
	buf[i] = '\0';
	return i;
}

int main() {
	char progpath[1024];
	{
		ssize_t len = readlink("/proc/self/exe", progpath, sizeof(progpath) - 1);
		if (len == -1) {
			const char msg[] = "error: failed to read full program path\n";
			write(STDERR_FILENO, msg, sizeof(msg) - 1);
			exit(EXIT_FAILURE);
		}

		while (progpath[len] != '/')
			--len;

		progpath[len] = '\0';
	}
```

```
char name1[256];
char name2[256];

int len1 = read_line(name1, sizeof(name1));
int len2 = read_line(name2, sizeof(name2));
if (len1 <= 1 || len2 <= 1) {
	const char msg[] = "error: need two file names\n";
	write(STDERR_FILENO, msg, sizeof(msg) - 1);
	exit(EXIT_FAILURE);
}
name1[len1 - 1] = '\0'; // убираем \n
name2[len2 - 1] = '\0';

int file1 = open(name1, O_WRONLY | O_CREAT | O_TRUNC, 0600);
int file2 = open(name2, O_WRONLY | O_CREAT | O_TRUNC, 0600);
if (file1 == -1 || file2 == -1) {
	const char msg[] = "error: failed to open file\n";
	write(STDERR_FILENO, msg, sizeof(msg) - 1);
	exit(EXIT_FAILURE);
}
int pipe1[2];
if (pipe(pipe1) == -1) {
	const char msg[] = "error: failed to create pipe\n";
	write(STDERR_FILENO, msg, sizeof(msg) - 1);
	exit(EXIT_FAILURE);
}

int pipe2[2];
if (pipe(pipe2) == -1) {
	const char msg[] = "error: failed to create pipe\n";
	write(STDERR_FILENO, msg, sizeof(msg) - 1);
	exit(EXIT_FAILURE);
}

const pid_t child1 = fork();

if (child1 == -1) {
	const char msg[] = "error: failed to spawn new process\n";
	write(STDERR_FILENO, msg, sizeof(msg) - 1);
	exit(EXIT_FAILURE);
}

if (child1 == 0) {
	close(pipe1[1]);
	close(pipe2[0]);
	close(pipe2[1]);
	close(file2);

	dup2(pipe1[0], STDIN_FILENO);
	close(pipe1[0]);

	dup2(file1, STDOUT_FILENO);
	close(file1);

	char path[1024];
	snprintf(path, sizeof(path) - 1, "%s/%s", progpath, CLIENT_PROGRAM_NAME);

	char *const args[] = {CLIENT_PROGRAM_NAME, NULL};
	execv(path, args);

	const char msg[] = "error: failed to exec into new exectuable image\n";
	write(STDERR_FILENO, msg, sizeof(msg) - 1);
	exit(EXIT_FAILURE);
}
```

```
	const pid_t child2 = fork();

	if (child2 == -1) {
		const char msg[] = "error: failed to spawn new process\n";
		write(STDERR_FILENO, msg, sizeof(msg) - 1);
		exit(EXIT_FAILURE);
	}

	if (child2 == 0) {
		close(pipe2[1]);
		close(pipe1[0]);
		close(pipe1[1]);
		close(file1);

		dup2(pipe2[0], STDIN_FILENO);
		close(pipe2[0]);

		dup2(file2, STDOUT_FILENO);
		close(file2);

		char path[1024];
		snprintf(path, sizeof(path) - 1, "%s/%s", progpath, CLIENT_PROGRAM_NAME);

		char *const args[] = {CLIENT_PROGRAM_NAME, NULL};
		execv(path, args);

		const char msg[] = "error: failed to exec into new exectuable image\n";
		write(STDERR_FILENO, msg, sizeof(msg) - 1);
		exit(EXIT_FAILURE);
	}

	close(pipe1[0]);
	close(pipe2[0]);
	close(file1);
	close(file2);

	int rnd = open("/dev/urandom", O_RDONLY);

	char buf[4096];
	int bytes;

	while ((bytes = read_line(buf, sizeof(buf))) > 0) {
		unsigned char r;
		read(rnd, &r, 1);

		// NOTE: 80% в pipe1, иначе в pipe2
		if (r % 100 < 80) {
			write(pipe1[1], buf, bytes);
		} else {
			write(pipe2[1], buf, bytes);
		}
	}

	close(rnd);
	close(pipe1[1]);
	close(pipe2[1]);

	wait(NULL);
	wait(NULL);
}
```

child.c

```
#include <stdint.h>
```

```
#include <stdbool.h>

#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>

bool is_vowel(char c) {
	return c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u' ||
		c == 'A' || c == 'E' || c == 'I' || c == 'O' || c == 'U';
}

int main() {
	char buf[4096];
	char out[4096];
	ssize_t bytes;

	while ((bytes = read(STDIN_FILENO, buf, sizeof(buf))) > 0) {
		int len = 0;
		for (int i = 0; i < bytes; ++i) {
			if (!is_vowel(buf[i])) {
				out[len++] = buf[i];
			}
		}

		int written = write(STDOUT_FILENO, out, len);
		if (written != len) {
			const char msg[] = "error: failed to write to file\n";
			write(STDERR_FILENO, msg, sizeof(msg) - 1);
			exit(EXIT_FAILURE);
		}
	}

	if (bytes < 0) {
		const char msg[] = "error: failed to read from stdin\n";
		write(STDERR_FILENO, msg, sizeof(msg) - 1);
		exit(EXIT_FAILURE);
	}
}
```

Протокол работы программы

Тестирование:

```
$ ./parent
file1.txt
file2.txt
Hello world
Operating systems
process and pipe
linux kernel
fork and exec
string six
lab number one
variant nineteen
parent child
student
$ cat < file1.txt
Hll wrld
prcss nd pp
lnx krnl
frk nd xc
strng sx
vrnt nntn
prnt chld
stdnt
```

```
$ cat < file2.txt
prtng systms
lb nmbr n
```

В приведенном запуске 8 из 10 строк попали в file1.txt (pipe1), а 2 строки – в file2.txt (pipe2), что соответствует заданной вероятности 80% / 20%. Во всех строках гласные удалены. Ввод завершен нажатием Ctrl-D.

Вывод

В ходе лабораторной работы была реализована система из родительского и двух дочерних процессов, обменивающихся данными через неименованные каналы (pipe). Родитель распределяет строки между каналами с вероятностью 80% / 20%, а дочерние процессы, запущенные через fork и execv с подменой stdin и stdout через dup2, удаляют из строк гласные и пишут результат в свои файлы. Основная сложность состояла в корректном закрытии неиспользуемых концов каналов: без этого дочерние процессы не получают конец файла и программа зависает на wait. Также пришлось отказаться от rand() и брать случайные числа из /dev/urandom через read, так как по условию разрешены только системные вызовы.